

Laboratorio 4

Programmazione di Sistema | A.A. 2025–26 | 13 e 15 Maggio 2026

Nota

Prima di iniziare l'esercizio 2, leggere il documento Adapter Pattern in Rust: Transforming One Iterator into Another fornito assieme al lab.

1 Esercizio 1 – Editor di testo

1.1 Descrizione

Una struct `LineEditor` racchiude lo stato di un editor di testo in cui il contenuto è memorizzato come vettore di stringhe, una per riga. Questo approccio è comune nelle implementazioni di editor: memorizzare tutto il testo in una singola stringa richiederebbe continue riallocazioni ad ogni modifica, rendendola inefficiente per file di grandi dimensioni. Suddividere il testo in righe minimizza la porzione di memoria che deve essere riallocata ad ogni operazione.

L'editor espone un'interfaccia semplice pensata per essere estesa tramite plugin:

- `all_lines()` restituisce un vettore di riferimenti a ciascuna riga (senza copiarle)
- `replace()` permette di sostituire una sottostringa di una riga

1.2 Sviluppo

Il file `editor.rs` contiene le interfacce delle struct da implementare. Il file non compila perché mancano i lifetime necessari. Il suggerimento è commentare tutto il codice, scommentare un blocco alla volta partendo dall'alto e aggiungere i lifetime prima di passare all'implementazione.

I test presenti nel file mostrano come le struct devono essere usate.

Passo 1 – LineEditor

Implementare `new()` e `from_file()`, che costruisce un `LineEditor` leggendo il contenuto da file. Implementare `replace()`, che sostituisce la sottostringa `[start..end]` della riga `line` con `subst`.

Passo 2 – Match e FindReplace

La struct `Match` contiene le informazioni su un'occorrenza trovata nel testo: numero di riga, offset di inizio e fine, testo corrispondente e sostituzione opzionale.

Aggiungere i lifetime mancanti a `Match` e a `FindReplace`, poi implementare:

- `new()` che compila il pattern con `regex` e cerca tutte le occorrenze
- `matches()` che restituisce la lista dei match trovati
- `apply()` che itera sui match e chiama la funzione passata per ciascuno, permettendo di accettarlo e impostare la sostituzione

Usare il crate `regex` e il metodo `find_iter` per trovare le occorrenze.

Domanda

Nel test `test_find_replace` sono mostrati due modi per applicare le sostituzioni all'editor. Perché il primo loop non funziona? Perché il secondo (con il vettore intermedio `subs`) funziona?

Passo 3 – LazyFinder

`FindReplace` raccoglie tutti i match prima di restituirli. Questo può essere costoso se il testo è grande. `LazyFinder` implementa una ricerca pigra: ogni chiamata a `next()` trova e restituisce solo il match successivo, salvando la posizione corrente per la chiamata seguente.

Implementare `new()` e `next()`.

Domanda

Cosa deve salvare `LazyFinder` tra una chiamata e l'altra di `next()`? Come si traduce questo in termini di campi della struct?

Passo 4 – FindIter

`LazyFinder` ha un metodo `next()` ma non implementa il trait `Iterator`. `FindIter` fa la stessa cosa implementando il trait correttamente, in modo da poter essere usato nei cicli `for` e concatenato con gli altri adapter della libreria standard.

Implementare `Iterator` per `FindIter`.

Domanda

Qual è la differenza tra il `next()` di `LazyFinder` e quello richiesto dal trait `Iterator`? Cosa si guadagna implementando il trait?

2 Esercizio 2 – Grep

2.1 Descrizione

L'esercizio è diviso in due parti. Nella prima si implementa un adapter su iteratori di interi per filtrare solo i numeri pari. Nella seconda si realizza un adapter che esegue l'operazione `grep` in modo ricorsivo su una directory.

Il file `grep.rs` contiene le interfacce e i test da completare.

2.2 Sviluppo

Passo 1 – EvenIter per i32

Implementare un adapter `EvenIter` che wrappa un iteratore su `i32` e restituisce solo i valori pari. Aggiungere i generic e i constraint necessari.

```
let v = vec![1, 2, 3, 4, 5];
for item in v.into_iter().even() {
    println!("{}", item); // stampa 2 e 4
}
```

Domanda

Nel test è usato `into_iter()` invece di `iter()`. Perché `iter()` non funzionerebbe qui?

Passo 2 – Aggiungere `even()` a tutti gli iteratori su `i32`

Definire un trait `AddEvenIter` con il metodo `even()` e fornire un'implementazione generica per tutti gli iteratori il cui `Item` è `i32`.

Passo 3 – EvenIter generico per tutti i tipi interi

Generalizzare `EvenIter` per funzionare con qualsiasi tipo intero usando il crate `num`:

```
cargo add num
```

Domanda

Il modulo `even_iter` definisce due parametri generici `I` e `U`. Quale è il significato di ciascuno e il motivo dei constraint imposti?

Passo 4 – GrepIter

Installare i crate necessari:

```
cargo add walkdir
cargo add regex
```

Implementare `GrepIter`, un adapter su `walkdir::IntoIter` che per ogni file trovato cerca le occorrenze di un pattern e restituisce un `Match` per ciascuna riga corrispondente. Gli errori di accesso ai file devono essere propagati come `Err`.

```
let walker = walkdir::WalkDir::new("/una_dir");
for m in walker.into_iter().grep(".*") {
    match m {
        Ok(m) => println!("{}", m.file, m.line, m.text),
        Err(e) => println!("errore: {}", e),
    }
}
```

Passo 5 - Aggiungere grep() a walkdir::IntoIter

Definire un trait `Grep` e implementarlo per `walkdir::IntoIter` in modo da poter chiamare `.grep(pattern)` direttamente sull'iteratore, come mostrato nell'esempio sopra.

Domanda

Se l'implementazione di `GrepIter` risulta complessa, come si può scomporla in più adapter concatenati? Quali vantaggi porta questa scelta?

3 Esercizio 3 – Albero di Natale

3.1 Descrizione

Realizzare una struct `Albero` che gestisca le luci di un albero di Natale. La struttura è un albero con una radice, senza cicli e senza link in avanti se non verso i figli. Ogni nodo ha un nome (stringa arbitraria), un interruttore e una luce che può essere accesa o spenta.

La luce di un nodo è accesa se e solo se il suo interruttore è `on` e tutti gli interruttori dei nodi che lo collegano alla radice sono `on`: basta un interruttore `off` sul percorso per spegnere la luce.

3.2 Interfaccia

```
impl Albero {  
    // aggiunge un nodo figlio del nodo father  
    pub fn add(&mut self, father: &str, node: &str) -> Result<(), ...> {}  
  
    // rimuove un nodo e tutti i suoi discendenti  
    pub fn remove(&mut self, node: &str) -> Result<(), ...> {}  
  
    // commuta l'interruttore del nodo e restituisce il nuovo valore  
    pub fn toggle(&mut self, node: &str) -> Result<bool, ...> {}  
  
    // restituisce se la luce del nodo e' accesa  
    pub fn peek(&self, node: &str) -> Result<bool, ...> {}  
}
```

3.3 Sviluppo

Passo 1 – Struttura dati

Attenzione

Non costruire la struttura dell'albero con una struct esplicita del tipo `Node { children: Vec<Node>, switch: bool }`. Memorizzare invece le relazioni tra nodi e il loro stato in collezioni della libreria standard costruite in modo opportuno.

Domanda

Quali collezioni della libreria standard sono adatte a rappresentare le relazioni padre-figlio tra nodi? Come si memorizza lo stato dell'interruttore di ciascun nodo?

Passo 2 – Implementare i metodi

Implementare i quattro metodi gestendo tutti i casi di errore:

- `add`: errore se `father` non esiste o `node` esiste già
- `remove`: errore se il nodo non esiste
- `toggle`: errore se il nodo non esiste
- `peek`: errore se il nodo non esiste; percorrere il cammino dalla radice al nodo per verificare che tutti gli interruttori siano `on`

Passo 3 - Test

Scrivere test che verifichino almeno:

- `test_add_remove` – aggiunta e rimozione di nodi con i loro discendenti
- `test_toggle_peek` – la luce si accende e spegne correttamente
- `test_propagation` – un interruttore `off` sul percorso spegne i discendenti
- `test_errors` – operazioni su nodi inesistenti restituiscono errore